Programming Assignment: Tomasulo Algorithm

Adding in-order multi-issue (1–4 instructions per cycle) to the Tomasulo simulator

Ilan Lipsky

1 Overview

The assignment starts from an existing Python simulator of Tomasulo's algorithm with a reorder buffer (ROB), register alias tables (RAT) and a load/store queue (LSQ), and asks for support for issuing $W = 1$–4 instructions per cycle, in program order, with the issue width given in the input file. In summary, I:

- added **in-order multi-issue** (issue width 1–4, default 1), a **multi-commit** stage (commit width, defaults to the issue width) and a configurable number of **CDBs** (default 1, as in the handout);
- implemented the **branch unit** the handout specifies but the original code lacked: 8-entry BTB indexed by the 3 low bits of the PC word address, a 1-bit predictor per entry, prediction in ISSUE, resolution at the end of EX, and **misprediction recovery** (RAT recovery, RS/LSQ/FU/CDB-queue clean-up, ROB squash, correct fetch at cycle $n+2$). Without it, multi-issue past a branch is impossible;
- made the hardware **fully parameterizable from the input file** (RS counts, EX/MEM latencies, # of FUs, ROB size, issue/commit/CDB width, initial registers and memory), added `Beq` and `Sub/Sub.d`, and print the required final **register and non-zero memory** values;
- built a **verification harness**: a sequential golden model, timing-invariant checks, 9 directed test benches $\times$ 4 issue widths $\times$ 2 CDB settings, and 20,000 random differential runs. All pass.

2 Starting point

The provided code (`code/*.py`, Qing, 2017) simulates single-issue Tomasulo with speculation-free branch handling. Reading and running it showed the following limitations that shaped the work:

- **One instruction per cycle** everywhere: one issue, one CDB latch, at most one commit (plus a special case for a store followed by one more instruction).
- **No branch prediction**: at every `Bne` fetch is switched off (`PC.valid = 0`) until the branch resolves. `Beq` is not supported, and the branch offset is treated as a *byte* offset (`offset/4`) while the ISA defines `PC+4+offset«2`.
- **Hard-coded configuration**: RS sizes, latencies, ROB size and initial values are constants in `main.py`; `test_case.txt` is never read. Registers and memory are not printed, and branches are printed with an empty commit column.
- Records are `namedtuple` *classes* whose class attributes are mutated, and `mem.py` has latent bugs (`if (check_if_sd_committable):` tests the function object instead of calling it; that function reads a non-existent field `ldsd_tag`; the load-dispatch test checks `ld_sd_queue[0].data` instead of the entry being considered).

Since nearly every function had to accept W instructions per cycle and cope with squashed speculative state, I kept the original module structure (`init`, `issue`, `exe`, `mem`, `wb`, `commit`, `print_status`, `main`), but rewrote the bodies around small explicit record classes.

3 What was added to the code

File	Changes
`init.py`	Input-file parser (hardware table, `Issue width`, `Commit width`, `CDB width`, `ROB entries`, registers, memory, program; case-insensitive, commas optional); record classes `ROBEntry`, `RSEntry`, `LSQEntry`, `FU`; `Machine` state; `squash_after()` misprediction recovery.
`issue.py`	Issue up to W instructions per cycle in program order; stop at the first structural hazard (ROB, RS or LSQ full) and after a predicted-taken branch; BTB lookup for branches; no issue during misprediction recovery.
`predictor.py`	*New.* 8-entry BTB (index = word address `& 7`, tag = PC) with target and 1-bit taken/not-taken state; miss ⇒ predict not-taken.
`exe.py`	Several FUs per class; pipelined FP adder/multiplier (one new op per cycle each), unpipelined integer and address adders; oldest-ready-first selection; `Beq/Bne` resolution, BTB update and squash.
`mem.py`	Conservative disambiguation, store-to-load forwarding (1 cycle), single-ported non-pipelined memory, LSQ entry freed when a load gets its value.
`wb.py`	Up to `CDB width` broadcasts per cycle, oldest instruction first.
`commit.py`	Up to `Commit width` in-order commits per cycle; stores write memory at commit and hold the single memory port; memory-fault reporting for committed instructions only.
`print_status.py`	Required output: instruction status table, R0–R31, F0–F31, non-zero memory, plus a summary (cycles, IPC, mispredictions, squashed instructions).
`main.py`	Cycle loop, command-line options (`-w`, `-cdb`, `-commit`, `-v` cycle trace, `-btb`).
`tools/`	*New.* Golden model, invariant checks, test runner, random fuzzer.

Table 1: Summary of the changes by file.

3.1 Multi-issue (ISSUE stage)

Each cycle the issue stage fetches from the PC and issues up to W instructions *in order*. For each one it needs a free ROB entry and a free entry in its reservation station (integer adder for `Add`, `Addi`, `Sub`, `Beq` and `Bne`; FP adder, FP multiplier) or in the LSQ (`Ld/Sd`). The first instruction that cannot get its entries stops issue for the cycle, so a younger instruction never overtakes it. Sources are read through the RAT *before* the destination is renamed, so same-cycle dependences inside an issue group (e.g. `Addi R1,R1,8` followed by `Ld F1,0(R1)`) are renamed correctly: the second instruction waits on the first one's ROB tag. A branch is predicted with the BTB at issue. If it is predicted taken, the fetch group ends and the target is fetched next cycle; otherwise issue continues down the fall-through path in the same cycle.

3.2 Wider back end

Issuing W instructions per cycle only helps if the rest of the machine keeps up, so the back end is widened as well:

- **Commit width** (default = W): up to that many ROB-head instructions retire per cycle.

- **CDB width** (default 1, the handout's "one CDB"): number of results broadcast per cycle. Results waiting for a bus are picked oldest-first. Section 7 reports both 1 CDB and W CDBs, because with one bus the CDB becomes the bottleneck.
- **Multiple functional units** per class (the "# of FUs" column), each choosing the oldest ready instruction from its reservation stations.

3.3 Branch prediction and speculation

Branches go to the integer-adder reservation stations and use the integer adder to compare. At the end of EX (cycle n) the branch updates its BTB entry. If the actual next PC differs from the predicted one, `squash_after()` removes every younger instruction from the ROB, the reservation stations (including their wait-for tags), the LSQ, the functional units and the CDB queue. It then rebuilds the RAT from the surviving ROB entries. Cycle $n+1$ is the recovery cycle, and fetch from the correct path starts in cycle $n+2$, as the handout requires (e.g. `test1`, $W=1$: the `Bne` resolves in cycle 10 and the next `Ld` issues in cycle 12). Wrong-path instructions never touch architectural state, because registers are written only at commit and stores write memory only at commit. A wrong-path load that faults (bad address) is flagged and only reported if it commits.

3.4 Load/store queue

Loads and stores enter the LSQ in program order and compute their address on a dedicated address adder. In MEM, a load waits while any older store address is still unknown (conservative disambiguation, as in the original code). Otherwise it forwards from the nearest older store with the same address in 1 cycle, once that store's data is available, or else accesses the single-ported, non-pipelined memory for `Cycles in Mem` cycles. Stores are dequeued at commit, when they write memory and occupy the port for `Cycles in Mem` cycles (shown as their MEM interval). A store that commits in a cycle can still forward to a load in that same cycle. I found this case through `test4`: before handling it, `test4` ran *slower* at $W=2$ (30 cycles) than at $W=1$ (27), because a store that committed earlier under wider commit could no longer forward its value.

4 Timing model

Stage	Rule (cycle numbers as printed in the tables)
ISSUE	$\leq W$ per cycle, in order; resources freed earlier in the same cycle may be reused.
EX	starts $\geq$ ISSUE$+1$ and $\geq$ (last operand's WB)$+1$; lasts `Cycles in EX`.
MEM (Ld)	starts $\geq$ EX end$+1$; `Cycles in Mem` cycles, or 1 cycle if forwarded.
WB	$\geq$ EX end$+1$ (ALU) or MEM end$+1$ (Ld); $\leq$ `CDB width` per cycle. Stores and branches have no WB.
COMMIT	in order, $\leq$ commit width per cycle; $\geq$ WB$+1$ (ALU, Ld), $\geq$ EX end$+1$ (branch); a store commits once address and data are known and the memory port is free.
Mispredict	detected at EX end n; recovery in $n+1$; correct-path ISSUE from $n+2$.

Table 2: Pipeline timing rules. Each cycle evaluates COMMIT, WB, MEM, EX, ISSUE; all hand-offs are time-stamped so an instruction never passes two dependent stages in one cycle.

5 Verification

Each run is checked automatically (`tools/checks.py`):

1. **Functional**: final R0–R31, F0–F31 and all 256 memory cells must equal those of an independent sequential interpreter (`tools/reference.py`), and the committed instruction stream must equal the interpreter's dynamic trace (so no wrong-path instruction commits and none is lost).
2. **Timing invariants**: in-order issue and commit; $\leq W$ issued, $\leq$ commit-width committed and $\leq$ CDB-width broadcast per cycle; ISSUE $<$ EX $<$ (MEM) $<$ WB $<$ COMMIT; correct EX/MEM lengths; every consumer starts EX after its producer's WB; stores commit after their data is broadcast; one memory access at a time.

`tools/run_tests.py` runs the 9 test benches at $W = 1..4$ with one CDB and with W CDBs (72 runs). `tools/fuzz.py` generates random programs (ALU ops, aliasing loads/stores, forward branches, counted loops) on random machines (ROB 2–64, 1–4 RS, 1–2 FUs, varied latencies, CDB 1–4) and runs each at all four widths. Two seeds, 5,000 programs and 20,000 runs, exercised about 52,000 mispredictions, 96,000 squashed instructions and 15,000 forwards with **0 failures**. To check that the checker itself catches errors, I injected bugs on purpose: skipping RAT recovery deadlocks, broadcasting more than the CDB width trips the CDB invariant, and forwarding before store data is ready fails both `test4` and the fuzzer.

6 Test cases

Test	Purpose
`test0`	Configuration from the handout (`Issue width = 2`). The handout's 3-line program loops forever (`R2`$\neq$`R3`), so a decrement of R2 was added: the loop runs twice, and both branches mispredict (cold BTB, then loop exit).
`test1`	The loop shipped with the original code: Ld/Mult/Ld/Add/Sd over 3 elements, with `Bne` offset -8 (words). Exercises speculation, LSQ capacity and memory.
`test2`	16 independent int/FP instructions on 2 units of each type: best case for multi-issue.
`test3`	One RAW chain with WAR/WAW name dependences: width cannot help; renaming must be right.
`test4`	Store-to-load forwarding, a load stalled behind a store with unknown address, a load to a different address, two stores to the same address.
`test5`	Nested loops with `Beq`/`Bne` (3×2 iterations), an unconditional `Beq R0,R0`, and wrong-path code that would corrupt R9/F9/memory if squashing failed.
`test6`	Structural hazards: 4-entry ROB, 1–2 RS per unit, unpipelined 2-cycle integer adder.
`test7`, `test8`	"Test Case #01/#02" from the sample report, for comparison.

7 Results under different issue widths

All runs below finish with the same, correct registers and memory at every width (Section 7.7). Full outputs for every test, width and CDB setting are in `results/`.

		CDB width = 1				CDB width = W			Best
Test	Instr.	W=1	W=2	W=3	W=4	W=2	W=3	W=4	speedup
`test0`	8	19	15	13	13	15	13	13	1.46×
`test1`	25	57	52	51	51	52	51	51	1.12×
`test2`	16	22	19	19	19	14	12	10	2.20×
`test3`	10	31	28	27	27	28	27	26	1.19×
`test4`	13	27	23	23	22	23	23	22	1.23×
`test5`	37	71	56	56	54	54	53	53	1.34×
`test6`	10	28	28	28	28	28	28	28	1.00×
`test7`	17	36	35	35	35	33	33	33	1.09×
`test8`	17	47	46	46	46	46	46	46	1.02×

Table 3: Total cycles. *Best speedup* = cycles at W=1 / fewest cycles at any width and CDB setting.

	CDB width = 1				CDB width = W			Mispred. /
Test	W=1	W=2	W=3	W=4	W=2	W=3	W=4	squashed (W=1)
`test0`	0.42	0.53	0.62	0.62	0.53	0.62	0.62	2 / 2
`test1`	0.44	0.48	0.49	0.49	0.48	0.49	0.49	2 / 1
`test2`	0.73	0.84	0.84	0.84	1.14	1.33	1.60	0 / 0
`test3`	0.32	0.36	0.37	0.37	0.36	0.37	0.38	0 / 0
`test4`	0.48	0.57	0.57	0.59	0.57	0.57	0.59	0 / 0
`test5`	0.52	0.66	0.66	0.69	0.69	0.70	0.70	8 / 14
`test6`	0.36	0.36	0.36	0.36	0.36	0.36	0.36	0 / 0
`test7`	0.47	0.49	0.49	0.49	0.52	0.52	0.52	0 / 0
`test8`	0.36	0.37	0.37	0.37	0.37	0.37	0.37	0 / 0

Table 4: IPC (committed instructions / cycles), and branch mispredictions / squashed instructions at W=1.

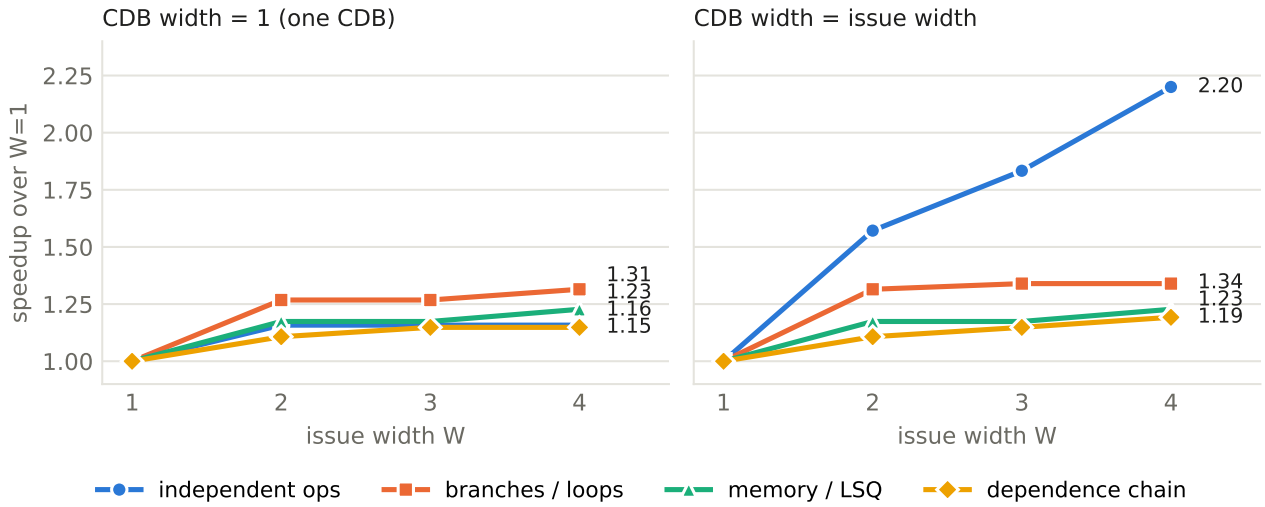


Figure 1: Speedup over single issue for four representative tests. Left: one CDB; right: one CDB per issue slot.

Discussion. Multi-issue always helps or is neutral; no test gets slower as W grows. How much it helps depends on what limits the program:

- **Independent instructions** (`test2`): 22 → 10 cycles at W=4 with 4 CDBs (IPC 0.73 → 1.60). With a single CDB the same machine stops at 19 cycles, because at most one result per cycle can be written back.
- **True dependences** (`test3`) and **long-latency chains** (`test1`, `test7`, `test8`, which are dominated by 15–20 cycle multiplies feeding adds) gain only 2–19%. Wider issue just fills the reservation stations earlier.
- **Branches** (`test0`, `test5`): each misprediction still costs the n+2 restart, but more instructions issue between restarts, so the gain is 31–46%.
- **Structural limits** (`test6`): the 4-entry ROB caps the window, so all widths take 28 cycles. Issue stalls in order whenever the ROB or an RS is full.
- **Sample-report cases**: at W=1, `test7` takes 36 cycles (sample report: 36), and at W=2..4 with W CDBs it takes 33 (sample: 33). `test8` takes 47/46 cycles (sample: 47/46). Individual cycles differ slightly because CDB arbitration here is oldest-first rather than FIFO.

7.1 Test 0: handout configuration, $W = 1..4$

At W=4 the whole loop body issues in cycle 1. The first `Bne` (cold BTB, predicted not-taken, actually taken) resolves in cycle 4 and the loop restarts in cycle 6. The second `Bne` hits in the BTB and is predicted taken, but the loop exits, so it mispredicts again.

W = 1

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Add.d F1, F2, F3`	1	2–4	–	5	6
`Ld F4, 8(R1)`	2	3–3	4–7	8	9
`Addi R2, R2, -10`	3	4–4	–	6	10
`Bne R2, R3, -4`	4	7–7	–	–	11
`Add.d F1, F2, F3`	9	10–12	–	13	14
`Ld F4, 8(R1)`	10	11–11	12–15	16	17
`Addi R2, R2, -10`	11	12–12	–	14	18
`Bne R2, R3, -4`	12	15–15	–	–	19

W = 2

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Add.d F1, F2, F3`	1	2–4	–	5	6
`Ld F4, 8(R1)`	1	2–2	3–6	7	8
`Addi R2, R2, -10`	2	3–3	–	4	8
`Bne R2, R3, -4`	2	5–5	–	–	9
`Add.d F1, F2, F3`	7	8–10	–	11	12
`Ld F4, 8(R1)`	7	8–8	9–12	13	14
`Addi R2, R2, -10`	8	9–9	–	10	14
`Bne R2, R3, -4`	8	11–11	–	–	15

W = 3

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Add.d F1, F2, F3`	1	2–4	–	5	6
`Ld F4, 8(R1)`	1	2–2	3–6	7	8
`Addi R2, R2, -10`	1	2–2	–	3	8
`Bne R2, R3, -4`	2	4–4	–	–	8
`Add.d F1, F2, F3`	6	7–9	–	10	11
`Ld F4, 8(R1)`	6	7–7	8–11	12	13
`Addi R2, R2, -10`	6	7–7	–	8	13
`Bne R2, R3, -4`	7	9–9	–	–	13

W = 4

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Add.d F1, F2, F3`	1	2–4	–	5	6
`Ld F4, 8(R1)`	1	2–2	3–6	7	8
`Addi R2, R2, -10`	1	2–2	–	3	8
`Bne R2, R3, -4`	1	4–4	–	–	8
`Add.d F1, F2, F3`	6	7–9	–	10	11
`Ld F4, 8(R1)`	6	7–7	8–11	12	13
`Addi R2, R2, -10`	6	7–7	–	8	13
`Bne R2, R3, -4`	6	9–9	–	–	13

Table 5: `test0` instruction status (1 CDB). EX/MEM show first–last cycle; "–" = stage not used.

7.2 Test 2: independent instructions

	W=1		W=2		W=3		W=4	
	I	C	I	C	I	C	I	C
`Add R10, R1, R2`	1	4	1	4	1	4	1	4
`Add.d F10, F1, F2`	2	6	1	5	1	5	1	5
`Sub R11, R3, R4`	3	7	2	5	1	5	1	5
`Mult.d F11, F3, F4`	4	10	2	8	2	8	1	7
`Addi R12, R5, 100`	5	11	3	8	2	8	2	7
`Sub.d F12, F5, F6`	6	12	3	9	2	8	2	7
`Add R13, R6, R1`	7	13	4	9	3	9	2	7
`Add.d F13, F2, F3`	8	14	4	10	3	9	2	8
`Sub R14, R2, R5`	9	15	5	10	3	9	3	8
`Mult.d F14, F5, F6`	10	16	5	11	4	10	3	9
`Addi R15, R3, -7`	11	17	6	11	4	10	3	9
`Sub.d F15, F1, F4`	12	18	6	12	4	10	3	9
`Add R16, R4, R6`	13	19	7	12	5	11	4	9
`Add.d F16, F4, F5`	14	20	7	13	5	11	4	10
`Addi R17, R1, 42`	15	21	8	13	5	11	4	10
`Mult.d F17, F1, F6`	16	22	8	14	6	12	4	10
Total	22 cycles		14 cycles		12 cycles		10 cycles	

Table 6: `test2`, ISSUE (I) and COMMIT (C) cycle at each width with CDB width $= W$. Exactly W instructions issue per cycle.

	W=1		W=2		W=3		W=4	
	I	C	I	C	I	C	I	C
`Add R10, R1, R2`	1	4	1	4	1	4	1	4
`Add.d F10, F1, F2`	2	6	1	5	1	5	1	5
`Sub R11, R3, R4`	3	7	2	6	1	6	1	6
`Mult.d F11, F3, F4`	4	10	2	8	2	8	1	7
`Addi R12, R5, 100`	5	11	3	8	2	8	2	8
`Sub.d F12, F5, F6`	6	12	3	9	2	9	2	9
`Add R13, R6, R1`	7	13	4	10	3	10	2	10
`Add.d F13, F2, F3`	8	14	4	11	3	11	2	11
`Sub R14, R2, R5`	9	15	5	12	3	12	3	12
`Mult.d F14, F5, F6`	10	16	5	13	4	13	3	13
`Addi R15, R3, -7`	11	17	6	14	4	14	3	14
`Sub.d F15, F1, F4`	12	18	6	15	4	15	3	15
`Add R16, R4, R6`	13	19	7	16	5	16	4	16
`Add.d F16, F4, F5`	14	20	7	17	5	17	4	17
`Addi R17, R1, 42`	15	21	8	18	5	18	4	18
`Mult.d F17, F1, F6`	16	22	8	19	6	19	4	19
Total	22 cycles		19 cycles		19 cycles		19 cycles	

Table 7: `test2` with a single CDB: issue still reaches W per cycle, but write-back serializes, so commits spread out.

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Add R10, R1, R2`	1	2–2	–	3	4
`Add.d F10, F1, F2`	1	2–3	–	4	5
`Sub R11, R3, R4`	1	2–2	–	3	5
`Mult.d F11, F3, F4`	1	2–5	–	6	7
`Addi R12, R5, 100`	2	3–3	–	4	7
`Sub.d F12, F5, F6`	2	3–4	–	5	7
`Add R13, R6, R1`	2	3–3	–	4	7
`Add.d F13, F2, F3`	2	3–4	–	5	8
`Sub R14, R2, R5`	3	4–4	–	5	8
`Mult.d F14, F5, F6`	3	4–7	–	8	9
`Addi R15, R3, -7`	3	4–4	–	5	9
`Sub.d F15, F1, F4`	3	4–5	–	6	9
`Add R16, R4, R6`	4	5–5	–	6	9
`Add.d F16, F4, F5`	4	5–6	–	7	10
`Addi R17, R1, 42`	4	5–5	–	6	10
`Mult.d F17, F1, F6`	4	5–8	–	9	10

Table 8: `test2`, full status at W=4, 4 CDBs. Up to 4 operations start EX per cycle on the two integer adders and the pipelined FP units, and 4 instructions commit per cycle.

7.3 Test 1: loop with speculation, $W = 1$ vs. $W = 4$

At W=4 an iteration issues in two cycles. The first `Bne` mispredicts (cold BTB) and resolves in cycle 5, so the next iteration issues in cycle 7. The second iteration's `Bne` is then correctly predicted taken. From the third iteration on, issue is throttled by the 5-entry LSQ, because stores stay in it until they commit. The two multiplies (15 cycles) dominate the run: 57 $\rightarrow$ 51 cycles.

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Ld F2, 0(R1)`	1	2–2	3–7	8	9
`Mult.d F4, F2, F20`	2	9–23	–	24	25
`Ld F6, 0(R2)`	3	4–4	8–12	13	26
`Add.d F6, F4, F6`	4	25–28	–	29	30
`Sd F6, 0(R2)`	5	6–6	34–38	–	34
`Addi R1, R1, -4`	6	7–7	–	9	35
`Addi R2, R2, -4`	7	8–8	–	10	36
`Bne R1, R0, -8`	8	10–10	–	–	37
`Ld F2, 0(R1)`	12	13–13	14–18	19	38
`Mult.d F4, F2, F20`	13	20–34	–	35	39
`Ld F6, 0(R2)`	14	15–15	19–23	25	40
`Add.d F6, F4, F6`	15	36–39	–	40	41
`Sd F6, 0(R2)`	16	17–17	42–46	–	42
`Addi R1, R1, -4`	17	18–18	–	20	43
`Addi R2, R2, -4`	18	19–19	–	21	44
`Bne R1, R0, -8`	19	21–21	–	–	45
`Ld F2, 0(R1)`	20	21–21	24–28	30	46
`Mult.d F4, F2, F20`	21	31–45	–	46	47
`Ld F6, 0(R2)`	22	23–23	29–33	34	48
`Add.d F6, F4, F6`	23	47–50	–	51	52
`Sd F6, 0(R2)`	24	25–25	53–57	–	53
`Addi R1, R1, -4`	25	26–26	–	27	54
`Addi R2, R2, -4`	26	27–27	–	28	55
`Bne R1, R0, -8`	27	28–28	–	–	56
`Add.d F20, F2, F2`	30	31–34	–	36	57

Table 9: `test1` instruction status, W=1 (1 CDB).

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Ld F2, 0(R1)`	1	2–2	3–7	8	9
`Mult.d F4, F2, F20`	1	9–23	–	24	25
`Ld F6, 0(R2)`	1	3–3	8–12	13	25
`Add.d F6, F4, F6`	1	25–28	–	29	30
`Sd F6, 0(R2)`	2	4–4	33–37	–	33
`Addi R1, R1, -4`	2	3–3	–	4	33
`Addi R2, R2, -4`	2	4–4	–	5	33
`Bne R1, R0, -8`	2	5–5	–	–	33
`Ld F2, 0(R1)`	7	8–8	13–17	18	34
`Mult.d F4, F2, F20`	7	19–33	–	34	35
`Ld F6, 0(R2)`	7	9–9	18–22	23	35
`Add.d F6, F4, F6`	7	35–38	–	39	40
`Sd F6, 0(R2)`	8	10–10	40–44	–	40
`Addi R1, R1, -4`	8	9–9	–	10	40
`Addi R2, R2, -4`	8	10–10	–	11	40
`Bne R1, R0, -8`	8	11–11	–	–	41
`Ld F2, 0(R1)`	12	13–13	23–27	28	41
`Mult.d F4, F2, F20`	12	29–43	–	44	45
`Ld F6, 0(R2)`	17	18–18	28–32	33	45
`Add.d F6, F4, F6`	17	45–48	–	49	50
`Sd F6, 0(R2)`	22	23–23	50–54	–	50
`Addi R1, R1, -4`	22	23–23	–	25	50
`Addi R2, R2, -4`	22	24–24	–	26	50
`Bne R1, R0, -8`	22	26–26	–	–	51
`Add.d F20, F2, F2`	28	29–32	–	35	51

Table 10: `test1` instruction status, W=4 (1 CDB).

7.4 Test 4: memory disambiguation and forwarding

`Ld F4,0(R1)` gets its value from `Sd F3,0(R1)` by forwarding (1-cycle MEM) as soon as the multiply result is broadcast. `Ld F7,24(R0)` forwards from the *nearest* older store to address 24 (`Sd F1,8(R2)`), not from `Sd F6,8(R2)`. `Ld F8` and `Ld F9` wait until the address of `Sd F7,16(R4)` (which depends on `Add R4`) is known, then both forward from it.

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Mult.d F3, F1, F2`	1	2–7	–	8	9
`Sd F3, 0(R1)`	2	3–3	10–13	–	10
`Ld F4, 0(R1)`	3	4–4	9–9	10	11
`Ld F5, 8(R1)`	4	5–5	6–9	11	12
`Add.d F6, F4, F5`	5	12–13	–	14	15
`Sd F6, 8(R2)`	6	7–7	16–19	–	16
`Sd F1, 8(R2)`	7	8–8	20–23	–	20
`Ld F7, 24(R0)`	8	9–9	10–10	12	21
`Add R4, R1, R2`	9	10–10	–	13	22
`Sd F7, 16(R4)`	10	14–14	24–27	–	24
`Ld F8, 40(R0)`	11	12–12	15–15	16	25
`Ld F9, 0(R3)`	12	13–13	15–15	17	26
`Add.d F10, F8, F9`	13	18–19	–	20	27

Table 11: `test4` instruction status, W=1 (1 CDB).

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Mult.d F3, F1, F2`	1	2–7	–	8	9
`Sd F3, 0(R1)`	1	2–2	9–12	–	9
`Ld F4, 0(R1)`	2	3–3	9–9	10	11
`Ld F5, 8(R1)`	2	4–4	5–8	9	11
`Add.d F6, F4, F5`	3	11–12	–	13	14
`Sd F6, 8(R2)`	3	5–5	14–17	–	14
`Sd F1, 8(R2)`	4	6–6	18–21	–	18
`Ld F7, 24(R0)`	4	7–7	8–8	11	18
`Add R4, R1, R2`	5	6–6	–	7	19
`Sd F7, 16(R4)`	8	9–9	22–25	–	22
`Ld F8, 40(R0)`	8	10–10	12–12	14	22
`Ld F9, 0(R3)`	9	11–11	12–12	15	23
`Add.d F10, F8, F9`	9	16–17	–	18	23

Table 12: `test4` instruction status, W=2 (1 CDB).

7.5 Tests 3, 6 and 7: issue/commit cycles per width

	W=1		W=2		W=3		W=4	
	I	C	I	C	I	C	I	C
`Addi R2, R1, 1`	1	4	1	4	1	4	1	4
`Add R3, R2, R2`	2	6	1	6	1	6	1	6
`Sub R4, R3, R1`	3	8	2	8	1	8	1	8
`Add.d F3, F1, F2`	4	9	2	8	2	8	1	8
`Mult.d F4, F3, F3`	5	15	3	13	2	13	2	12
`Sub.d F5, F4, F1`	6	19	3	17	2	17	2	16
`Add.d F3, F5, F5`	7	23	4	21	3	21	2	20
`Mult.d F1, F3, F2`	8	29	4	27	3	27	2	26
`Addi R2, R4, 10`	9	30	5	27	3	27	3	26
`Add R4, R2, R2`	10	31	5	28	4	27	3	26
Total	31 cycles		28 cycles		27 cycles		26 cycles	

Table 13: `test3` (dependence chain, CDB width $=$ W): instructions issue earlier, but each waits for its predecessor.

	W=1		W=2		W=3		W=4	
	I	C	I	C	I	C	I	C
`Ld F3, 0(R1)`	1	7	1	7	1	7	1	7
`Ld F4, 4(R1)`	2	10	1	10	1	10	1	10
`Add.d F5, F3, F4`	3	14	2	14	1	14	1	14
`Mult.d F6, F5, F2`	4	21	2	21	2	21	1	21
`Add.d F7, F1, F2`	10	22	10	21	10	21	10	21
`Mult.d F8, F1, F1`	14	23	14	22	14	22	14	22
`Addi R2, R1, 4`	15	24	14	22	14	22	14	22
`Addi R3, R2, 4`	21	25	21	25	21	25	21	25
`Sd F6, 0(R3)`	22	26	21	26	21	26	21	26
`Sub R4, R3, R1`	23	28	22	28	22	28	22	28
Total	28 cycles		28 cycles		28 cycles		28 cycles	

Table 14: `test6` (4-entry ROB, 1 CDB): the same total cycle count at every width. Issue stalls in order on a full ROB or RS (e.g. `Add.d F7` waits until the single FP-adder reservation station frees in cycle 10).

	W=1		W=2		W=3		W=4	
	I	C	I	C	I	C	I	C
`Ld F1, 0(R1)`	1	10	1	10	1	10	1	10
`Mult.d F2, F1, F0`	2	16	1	16	1	16	1	16
`Add.d F3, F3, F2`	3	20	2	20	1	20	1	20
`Mult.d F2, F1, F1`	4	21	2	20	2	20	1	20
`Add.d F4, F2, F4`	5	22	3	21	2	21	2	21
`Addi R1, R1, 8`	6	23	3	21	2	21	2	21
`Ld F1, 0(R1)`	7	24	4	22	3	21	2	21
`Mult.d F2, F1, F0`	8	25	4	22	3	22	2	22
`Add.d F3, F3, F2`	9	28	5	26	3	26	3	26
`Mult.d F2, F1, F1`	10	29	5	26	4	26	3	26
`Add.d F4, F2, F4`	11	30	6	27	4	27	3	27
`Addi R1, R1, 8`	12	31	6	27	4	27	3	27
`Ld F1, 0(R1)`	13	32	7	28	5	27	4	27
`Mult.d F2, F1, F0`	14	33	10	28	10	28	10	28
`Add.d F3, F3, F2`	16	34	16	32	16	32	16	32
`Mult.d F2, F1, F1`	17	35	16	32	16	32	16	32
`Add.d F4, F2, F4`	18	36	17	33	17	33	17	33
Total	36 cycles		33 cycles		33 cycles		33 cycles	

Table 15: `test7` (sample report Test Case #01, CDB width $= W$).

7.6 Test 5: branches at $W = 2$

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Addi R2, R0, 2`	1	2–2	–	3	4
`Add.d F3, F3, F1`	1	2–3	–	4	5
`Addi R2, R2, -1`	2	4–4	–	5	6
`Bne R2, R0, -3`	2	6–6	–	–	7
`Add.d F3, F3, F1`	8	9–10	–	11	12
`Addi R2, R2, -1`	8	9–9	–	10	12
`Bne R2, R0, -3`	9	11–11	–	–	13
`Mult.d F4, F3, F2`	13	14–17	–	18	19
`Addi R1, R1, -1`	13	14–14	–	15	19
`Beq R1, R0, 1`	14	16–16	–	–	20
`Beq R0, R0, -8`	14	15–15	–	–	20
`Addi R2, R0, 2`	17	18–18	–	19	21
`Add.d F3, F3, F1`	17	18–19	–	20	21
`Addi R2, R2, -1`	18	20–20	–	21	22
`Bne R2, R0, -3`	18	22–22	–	–	23
`Add.d F3, F3, F1`	24	25–26	–	27	28
`Addi R2, R2, -1`	24	25–25	–	26	28
`Bne R2, R0, -3`	25	27–27	–	–	29
`Mult.d F4, F3, F2`	29	30–33	–	34	35
`Addi R1, R1, -1`	29	30–30	–	31	35
`Beq R1, R0, 1`	30	32–32	–	–	36
`Beq R0, R0, -8`	30	31–31	–	–	36
`Addi R2, R0, 2`	31	33–33	–	35	37
`Add.d F3, F3, F1`	31	32–33	–	36	37
`Addi R2, R2, -1`	32	36–36	–	37	38
`Bne R2, R0, -3`	32	38–38	–	–	39
`Add.d F3, F3, F1`	40	41–42	–	43	44
`Addi R2, R2, -1`	40	41–41	–	42	44
`Bne R2, R0, -3`	41	43–43	–	–	45
`Mult.d F4, F3, F2`	45	46–49	–	50	51
`Addi R1, R1, -1`	45	46–46	–	47	51
`Beq R1, R0, 1`	46	48–48	–	–	52
`Addi R9, R0, 7`	50	51–51	–	52	53
`Beq R9, R0, 2`	50	53–53	–	–	54
`Add.d F9, F3, F4`	51	52–53	–	54	55
`Sd F9, 200(R0)`	51	52–52	55–56	–	55
`Addi R10, R9, 1`	52	54–54	–	55	56

Table 16: `test5` at W=2 (1 CDB), committed instructions only. 29 wrong-path instructions were issued and squashed. Every gap in the ISSUE column after a branch resolves is the $n \to n{+}2$ restart.

7.7 Final register and memory values

These values are identical for every issue width and CDB setting, and equal to the sequential golden model's (non-zero values only):

- `test0_handout_example`. Registers: `R1=10`, `F1=30.1`, `F2=30.1`. Memory: `Mem[4]=1`, `Mem[8]=2`, `Mem[12]=3.4`.
- `test1_loop`. Registers: `R2=20`, `F2=3.0`, `F4=9.0`, `F6=15.0`, `F20=6.0`. Memory: `Mem[4]=3.0`, `Mem[8]=2.0`, `Mem[12]=1.0`, `Mem[24]=15.0`, `Mem[28]=11.0`, `Mem[32]=7.0`.
- `test2_independent`. Registers: `R1=1`, `R2=2`, `R3=3`, `R4=4`, `R5=5`, `R6=6`, `R10=3`, `R11=-1`, `R12=105`, `R13=7`, `R14=-3`, `R15=-4`, `R16=10`, `R17=43`, `F1=1.5`, `F2=2.5`, `F3=3.5`, `F4=4.5`, `F5=5.5`, `F6=6.5`, `F10=4.0`, `F11=15.75`, `F12=-1.0`, `F13=6.0`, `F14=35.75`, `F15=-3.0`, `F16=10.0`, `F17=9.75`. Memory: none.
- `test3_dependency_chain`. Registers: `R1=5`, `R2=17`, `R3=12`, `R4=34`, `F1=138.0`, `F2=3.0`, `F3=46.0`, `F4=25.0`, `F5=23.0`. Memory: none.
- `test4_memory`. Registers: `R1=8`, `R2=16`, `R3=40`, `R4=24`, `F1=1.5`, `F2=2.0`, `F3=3.0`, `F4=3.0`,

F5=20.0,F6=23.0,F7=1.5,F8=1.5,F9=1.5,F10=3.0. Memory: Mem[8]=3.0,Mem[16]=20.0, Mem[24]=1.5,Mem[40]=1.5.

- test5_branches. Registers: R9=7,R10=8,F1=1.0,F2=0.5,F3=6.0,F4=3.0,F9=9.0. Memory: Mem[200]=9.0.
- test6_structural. Registers: R1=4,R2=8,R3=12,R4=8,F1=1.0,F2=2.0,F3=7.0,F4=8.0, F5=15.0,F6=30.0,F7=3.0,F8=1.0. Memory: Mem[4]=7.0,Mem[8]=8.0,Mem[12]=30.0.
- test7_sample_report_tc1. Registers: R1=28,R2=32,F1=5.0,F2=25.0,F4=26.0,F20=3.0. Memory: Mem[4]=3.0,Mem[8]=2.0,Mem[12]=1.0,Mem[24]=6.0,Mem[28]=5.0,Mem[32]=4.0.
- test8_sample_report_tc2. Registers: R1=24,R2=32,F1=6.0,F2=36.0,F4=37.0,F20=3.0. Memory: Mem[4]=3.0,Mem[8]=2.0,Mem[12]=1.0,Mem[24]=6.0,Mem[28]=5.0,Mem[32]=4.0.

7.8 Example of the complete simulator output

`python3 main.py ../tests/test1_loop.txt -w 2`:

```
Configuration: issue width=2, commit width=2, CDB width=1, ROB=64

                    ISSUE   EX        MEM       WB      COMMIT
Ld F2, 0(R1)        1       2-2       3-7       8       9
Mult.d F4, F2, F20  1       9-23      -         24      25
Ld F6, 0(R2)        2       3-3       8-12      13      25
Add.d F6, F4, F6    2       25-28     -         29      30
Sd F6, 0(R2)        3       4-4       33-37     -       33
Addi R1, R1, -4     3       4-4       -         5       33
Addi R2, R2, -4     4       5-5       -         6       34
Bne R1, R0, -8      4       6-6       -         -       34
Ld F2, 0(R1)        8       9-9       13-17     18      35
Mult.d F4, F2, F20  8       19-33     -         34      35
Ld F6, 0(R2)        9       10-10     18-22     23      36
Add.d F6, F4, F6    9       35-38     -         39      40
Sd F6, 0(R2)        10      11-11     40-44     -       40
Addi R1, R1, -4     10      11-11     -         12      41
Addi R2, R2, -4     11      12-12     -         14      41
Bne R1, R0, -8      11      13-13     -         -       42
Ld F2, 0(R1)        12      13-13     23-27     28      42
Mult.d F4, F2, F20  12      29-43     -         44      45
Ld F6, 0(R2)        17      18-18     28-32     33      45
Add.d F6, F4, F6    17      45-48     -         49      50
Sd F6, 0(R2)        22      23-23     50-54     -       50
Addi R1, R1, -4     22      23-23     -         25      51
Addi R2, R2, -4     23      24-24     -         26      51
Bne R1, R0, -8      23      26-26     -         -       52
Add.d F20, F2, F2   28      29-32     -         35      52

REGISTER VALUES
        R0        R1        R2        R3        R4        R5        R6        R7
value:  0         0         20        0         0         0         0         0
        R8        R9        R10       R11       R12       R13       R14       R15
value:  0         0         0         0         0         0         0         0
        R16       R17       R18       R19       R20       R21       R22       R23
value:  0         0         0         0         0         0         0         0
        R24       R25       R26       R27       R28       R29       R30       R31
value:  0         0         0         0         0         0         0         0
        F0        F1        F2        F3        F4        F5        F6        F7
value:  0.0       0.0       3.0       0.0       9.0       0.0       15.0      0.0
        F8        F9        F10       F11       F12       F13       F14       F15
value:  0.0       0.0       0.0       0.0       0.0       0.0       0.0       0.0
        F16       F17       F18       F19       F20       F21       F22       F23
value:  0.0       0.0       0.0       0.0       6.0       0.0       0.0       0.0
        F24       F25       F26       F27       F28       F29       F30       F31
value:  0.0       0.0       0.0       0.0       0.0       0.0       0.0       0.0

NON-ZERO MEMORY VALUES
```

```
Addresses   Values
4           3.0
8           2.0
12          1.0
24          15.0
28          11.0
32          7.0

SUMMARY
  total cycles            : 52
  instructions committed  : 25
  IPC                     : 0.481
  instructions issued     : 26 (squashed 1)
  branches issued         : 3, mispredicted 2
```

8 What works and what does not

Works: issue width 1–4 (default 1) set in the input file or with `-w`; in-order issue with structural-hazard stalls; commit width and number of CDBs; multiple FUs per class; pipelined FP units and unpipelined integer/address adders; register renaming through RAT + ROB; BTB with 1-bit prediction, speculative execution past branches and full misprediction recovery with the $n+2$ restart; `Beq`/`Bne`; LSQ with conservative disambiguation, store-to-load forwarding and stores written at commit; R0 hard-wired to 0; the required output format. All 72 directed runs and 20,000 random runs match the golden model.

Limitations and modelling choices:

- Loads never bypass a store with an unknown address. This is conservative, so no memory-order violation recovery is needed. It matches the original code, but it is stricter than letting a load speculate past unresolved stores.
- Memory is the original 256-cell array indexed by byte address. Each address holds one value, so unaligned or overlapping word accesses are not modelled. Test benches use word-aligned addresses.
- The BTB is updated when a branch resolves in EX, so a wrong-path branch that resolves before an older mispredicted branch can update it. Real hardware has the same effect; it only changes timing, never results.
- A taken-predicted branch ends the issue group for that cycle. A wrong-path load already accessing memory keeps the port busy until its access ends (non-pipelined memory).
- The output lists committed instructions only. Squashed instructions are counted in the summary and visible with `-v`.

9 How to run

```
cd code
python3 main.py ../tests/test1_loop.txt          # issue width from the file
python3 main.py ../tests/test1_loop.txt -w 4     # override issue width (1-4)
python3 main.py ../tests/test2_independent.txt -w 4 --cdb 4 -v   # 4 CDBs + cycle trace
python3 ../tools/run_tests.py                    # all tests, W=1..4, verified
python3 ../tools/fuzz.py 2000                    # random differential testing
```

Requires Python 3.6+ only (no third-party packages). See `README.md` for the input format.